

CI/CD: Jenkins (Docker) → Nexus (Docker) — Step-by-step notes, troubleshooting, interview points, LinkedIn post

Screenshot / uploaded file (reference): `/mnt/data/e86fdda7-bbea-4d04-b7f5-094bf384dc95.png`

Summary

This document captures the full mini-project: setting up Jenkins (in Docker) to build a Java Spring MVC `.war` and push artifacts to a repository. Initially Artifactory was tried but required PostgreSQL; later Nexus (Sonatype Nexus3) was used and successfully received artifacts. The notes include step-by-step commands, errors encountered, root causes, fixes, and suggested wording for interview and LinkedIn.

Goal

- Create a simple CI pipeline that: checks out code → builds `.war` using Maven → deploys (`mvn deploy`) the artifact to an artifact repository (Nexus).
 - Jenkins runs inside Docker (local). Nexus runs inside Docker (local). Maven runs from Jenkins build.
-

Environment used (local)

- Host OS: (user environment) — local laptop/desktop
 - Docker Engine: local
 - Jenkins: `jenkins/jenkins:lts` (container)
 - Nexus: `sonatype/nexus3` (container)
 - Maven: configured inside Jenkins (Global Tool Configuration) and invoked in pipeline
-

Key commands used

```
# Start Nexus
docker run -d --name nexus -p 8081:8081 -v nexus-data:/nexus-data sonatype/
nexus3

# Start Jenkins (example)
docker run -d --name jenkins -p 8080:8080 -p 50000:50000
-v jenkins_home:/var/jenkins_home
```

```
-v /var/run/docker.sock:/var/run/docker.sock
jenkins/jenkins:lts

# Create docker network
docker network create devops-net

docker network connect devops-net jenkins
docker network connect devops-net nexus

# Exec into Jenkins container
docker exec -it jenkins /bin/bash

# Create Maven settings.xml inside Jenkins container
mkdir -p /var/jenkins_home/.m2
cat <<EOF > /var/jenkins_home/.m2/settings.xml
<settings>
  <servers>
    <server>
      <id>nexus</id>
      <username>admin</username>
      <password>YOUR_NEXUS_PASSWORD</password>
    </server>
  </servers>
</settings>
EOF
```

Problems encountered (chronological) & fixes

1) Tried JFrog Artifactory container — failed on startup

Error (from logs):

```
Cannot start the application with a database other than PostgreSQL.
DB Schema initialization manager failed to init.
```

Cause: Latest Artifactory OSS requires PostgreSQL as the backing DB. The Docker container attempted to start without a proper PostgreSQL backend. **Fix options:** - Run Artifactory with a PostgreSQL container (use docker-compose with `postgres` service and set `DB_TYPE=postgresql` + environment variables), or - Use JFrog Cloud (signup issues with business email), or - Use Nexus instead (quicker and simpler for learning).

Decided: Use Nexus OSS (sonatype/nexus3) which is simpler to run locally.

2) Nexus: locating initial admin password

Note: After starting Nexus, the admin password lives inside the container at `/nexus-data/admin.password`. **Command:**

```
docker exec -it nexus /bin/bash
cat /nexus-data/admin.password
```

Use that password to login to `http://<host-ip>:8081` as `admin` and set a new password.

3) Jenkins (running in Docker) -> Maven deploy could not reach Nexus (Connection refused)

Error:

```
Could not transfer metadata ... from/to nexus (http://localhost:8081/...):
Connect to localhost:8081 ... failed: Connection refused
```

Cause: Inside Jenkins container, `localhost` refers to the container itself, not the host machine or the Nexus container. Jenkins tried to contact `localhost:8081` and failed. **Fixes:** - Preferred: Put Jenkins and Nexus on the same Docker network (e.g. `devops-net`) and use the Nexus container name in URLs.

Commands:

```
docker network create devops-net
docker network connect devops-net jenkins
docker network connect devops-net nexus
```

Update `pom.xml` repository URL to:

```
http://nexus:8081/repository/maven-releases/
```

This resolves DNS to the Nexus container from Jenkins container.

Alternative: Use host IP address of machine (e.g. `http://192.168.1.7:8081`) but using Docker network with container names is cleaner.

4) 400 Bad Request while deploying (snapshot vs release / repo settings)

Error:

```
status: 400 Bad Request ... 0.0.1-SNAPSHOT ... maven-releases
```

Cause: Trying to deploy a `-SNAPSHOT` version to a *release* repository; Nexus rejects mismatched version policy.

Fix: Either - Create a snapshot hosted repo (`maven-snapshots`) and use its URL in `<snapshotRepository>` or - Change project version to remove `-SNAPSHOT` and deploy to release repo.

Also check: `Deployment policy` in repository should be `Allow redeploy` (or `Allow`), `Version policy` must match artifact type.

5) 400 Bad Request after fixing snapshot/release and groupId case

Error:

```
transfer failed ... status: 400 Bad Request
```

Extra checks & fixes: - **Lowercased groupId:** `com.cg.hdfc.webdummy` (Nexus enforces path conventions) - Confirm `pom.xml` `distributionManagement` uses correct repository URL (container name or host ip) - Confirm `settings.xml` server id matches `<id>` in `distributionManagement` (`nexus`) and correct credentials are used

Result: Replaced hostname with container ID / container name in URL -> artifact successfully uploaded and visible in Nexus.

Final working configuration (summary)

- Jenkins container and Nexus container connected to the same Docker network `devops-net`
- `pom.xml` `distributionManagement` uses URL: `http://nexus:8081/repository/maven-releases/` (or `maven-snapshots` for snapshot builds)
- Jenkins container has Maven settings with credentials at `/var/jenkins_home/.m2/settings.xml` referencing `<id>nexus</id>`
- GroupId and artifact coordinates use lowercase groupId
- Nexus repository settings: `maven-releases` created with `Version policy = Release`, `Deployment policy = Allow redeploy`

Commands / snippets to include in the interview answer

Create Docker network and attach containers

```
docker network create devops-net
docker network connect devops-net jenkins
docker network connect devops-net nexus
```

Maven settings (on Jenkins container)

```
<settings>
  <servers>
    <server>
      <id>nexus</id>
      <username>admin</username>
      <password>YOUR_NEXUS_PASSWORD</password>
    </server>
  </servers>
</settings>
```

DistributionManagement in `pom.xml` (release example)

```
<distributionManagement>
  <repository>
    <id>nexus</id>
    <url>http://nexus:8081/repository/maven-releases/</url>
  </repository>
</distributionManagement>
```

Minimal Jenkinsfile (mvn deploy)

```
pipeline {
  agent any
  tools { maven 'MAVEN_HOME'; jdk 'JAVA_HOME' }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build') { steps { sh 'mvn clean package' } }
    stage('Deploy') { steps { sh 'mvn deploy -DskipTests' } }
  }
}
```

Interview talking points (concise)

Use these bullets when asked about the project in interviews:

- Setup: "I used Docker to run Jenkins (CI server) and Nexus (artifact repository) locally to build and store Java `.war` artifacts."
- Challenge: "Initially I tried Artifactory but it failed on startup because it required PostgreSQL. Rather than spend time configuring a DB, I switched to Nexus OSS to keep the pipeline simple for learning."
- Networking issue: "Inside containers, `localhost` is container-local. I resolved networking by creating a Docker network and using Nexus container name (DNS) in `pom.xml`."
- Repo policy issue: "Received HTTP 400 when pushing a SNAPSHOT to a release repo — fixed by creating the correct snapshot repo or releasing with a non-SNAPSHOT version."
- Final result: "Automated build and deploy with Jenkinsfile, artifacts are visible in Nexus; used Maven `settings.xml` in Jenkins for credentials."

LinkedIn post (short + professional)

Yesterday I built a simple CI/CD setup locally: Jenkins (Docker) → Maven build → Sonatype Nexus (Docker) for artifact storage. Learned several practical lessons about container networking (`localhost` vs container DNS), Maven deploy configuration, and repository policies (snapshot vs release). This hands-on mini-project helped me understand real-world issues that surface in CI pipelines. If anyone wants the Jenkinsfile and setup steps, happy to share the repo and notes. #DevOps #CI_CD #Jenkins #Maven #Nexus

Short checklist to include in your repo README (quick copy)

1. Start Nexus: `docker run -d --name nexus -p 8081:8081 -v nexus-data:/nexus-data sonatype/nexus3`
2. Start Jenkins: `docker run -d --name jenkins -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts`
3. Create Docker network: `docker network create devops-net`
4. Connect containers: `docker network connect devops-net nexus` and `docker network connect devops-net jenkins`
5. In Jenkins container, create `/var/jenkins_home/.m2/settings.xml` with Nexus credentials (`id=nexus`).
6. Update `pom.xml` `distributionManagement` to use `http://nexus:8081/repository/...` (use `maven-snapshots` for `-SNAPSHOT` versions).
7. Create `maven-releases` and `maven-snapshots` in Nexus and ensure `Allow redeploy` is enabled if needed.
8. Add Jenkinsfile to repo and run pipeline.

Final notes & tips

- Always prefer container names / Docker networks for inter-container communication.
 - Use snapshot repositories for iterative development; release repositories for official builds.
 - Keep `settings.xml` secure: use Jenkins credentials plugin and inject secrets securely into builds when possible.
 - For production Artifactory usage, follow JFrog docs to set up PostgreSQL or use cloud offering.
-

Attachment

- Included screenshot reference at top: `/mnt/data/e86fdda7-bbea-4d04-b7f5-094bf384dc95.png`

Document created to be used for interview prep and LinkedIn posting. You can copy, paste, and edit the LinkedIn text from the "LinkedIn post" section.